

Constants and Formatting

Tuesday, 7 July 2026

9:27 AM

* Constants-:

constants are declared with the `const` keyword. They can't use the `:=` short declaration syntax.

```
const pi = 3.14159
```

Constants can be primitive types like strings, integers, booleans and floats. They cannot be more complex types like slices, maps and structs, which are types we will explain later.

+ Computed constants-:

Constants can be computed as long as the computation can happen at compile time.

e.g.

```
const fName: "Herhal"
const lastName: "Gonbotz"
const fullName: "fName" + " " + lastName
```

you cannot declare a constant that can only be computed at run-time.

```
const cTime: time.Now() // not working.
```

* Formatting strings in Go-:

Go follows the `printf` tradition from the C language.

- `fmt.Printf()` -: Prints a formatted string to standard out
- `fmt.Sprintf()` -: Returns the formatting string

-:- Default Representation-:

The `%v` variant print any value in a default format. It can be used as a catchall.

```
for string %s
for integer %d
for float %f
```

* Runs and string Encoding-:

In Go, strings are just sequence of bytes. they can hold arbitrary data. However, Go also has a special type `rune`, which is alias for `int32`.

This means that `rune` is 32-bit integer, which is large enough to hold any unicode code point.

When you're working with strings, you need to be aware of the encoding (bytes $\rightarrow$ representation). Go uses UTF-8 encoding, which is a variable-length encoding.

*:- What Does This Mean?

1. When you need to work with individual characters in a string, you should use the `rune` type. It breaks strings up into their individual characters, which can be more than one byte long.
2. We can include a wide variety of unicode characters in our strings, such as emojis and Chinese characters, and Go will handle them just fine.